Software Engineering Department
Braude College of Engineering

Final Project – Phase B (61999)

A Multimodal Deep Learning System for Predicting Song Popularity and Suggesting Audio Enhancements

CODE: 26-1-D-5

By:
Arad Harush 211879606
Shahar Nahum 323964817

Advisor:
Dr. Dan Lemberg | lembergdan@braude.ac.il
Mrs. Elena Kramer | elenak@braude.ac.il

Link to GitHub:
https://github.com/aradharush/Song-Popularity-Prediction-System

1 Abstract

The modern music industry is characterized by an unprecedented volume of daily releases, making the identification of potentially successful songs a significant challenge for producers, artists, and record labels. Recent advances in Music Information Retrieval (MIR) have applied deep learning to song-popularity prediction, yet many existing approaches focus on a single data modality or provide predictions without actionable recommendations. This project presents a dual-stage framework designed to connect popularity prediction with interpretable song-optimization recommendations.

2 Introduction

The contemporary music industry is undergoing a paradigm shift driven by the rapid digitization of distribution channels. Streaming platforms have democratized music creation, resulting in an unprecedented volume of daily releases, estimated at over 100,000 new tracks every single day. This saturation has transformed the marketplace into a highly competitive environment in which identifying tracks with strong commercial potential has become an important objective for record labels, producers, and independent artists alike. Historically, the process of identifying commercially viable tracks relied heavily on the subjective intuition of A&R (Artists and Repertoire) professionals or basic statistical analysis of metadata. However, these traditional methods are increasingly insufficient in capturing the complex, stochastic nature of musical preference in the modern era, creating a pressing need for objective, data-driven analytical tools.

While the field of Music Information Retrieval (MIR) has seen significant advancements with the advent of Deep Learning, the current landscape of popularity prediction remains fragmented. A fundamental challenge lies in the multi-modal nature of music perception; the appeal of a song is derived from an intricate interplay between its sonic texture, rhythmic energy, and the semantic resonance of its lyrics. Many existing computational approaches address only part of this complexity, often focusing on audio signal processing or textual analysis in isolation. Furthermore, the industry faces a critical "interpretability gap." Conventional predictive models operate primarily as discriminative "black boxes"—they may successfully classify a song's

potential for success, but they offer no transparency regarding the underlying factors driving that decision. For a content creator, merely knowing that a track is predicted to achieve a low score is of limited value without actionable insights on how to rectify it.

To address these challenges, this project uses a dual-phase pipeline consisting of Perception and Optimization. In the Perception phase, a multimodal neural network processes three data sources: raw audio, lyrical content, and temporal metadata represented by the intended release year. Each source contributes different information to the final popularity estimate. In the Optimization phase, the system acts as a "Virtual Producer" by analyzing the track's acoustic-feature profile and suggesting targeted adjustments, such as changes to energy or loudness. The goal is not only to output a score, but also to provide the user with understandable directions for possible improvement.

3 Literature Review

The domain of Music Information Retrieval (MIR) has evolved significantly over the past decade, transitioning from handcrafted feature extraction to end-to-end Deep Learning (DL) methodologies. This section reviews the theoretical foundations and state-of-the-art technologies relevant to multi-modal popularity prediction and algorithmic optimization via Reinforcement Learning.

3.1 Multimodal Hit Song Prediction

Predicting the commercial success of music is a complex task due to the stochastic nature of public preference. While early approaches relied heavily on metadata or singular modalities, recent research demonstrates that combining auditory and textual data yields superior predictive performance. A pivotal study in this domain is the work of Castelli [1], which investigated the efficacy of combining audio embeddings with lyric embeddings. Castelli's research highlights that a "Late Fusion" architecture—where features are extracted independently from audio and text before being concatenated—significantly outperforms uni-modal baselines. This finding aligns with broader research by Oramas et al. [2], who established that multi-modal networks integrating audio, text, and images provide a more comprehensive representation of musical

content, leading to higher accuracy in classification and recommendation tasks.

3.2 Audio Analysis: CNNs and Spectrograms

To process raw audio data using deep learning, time-domain signals are typically converted into time-frequency representations, such as Mel-spectrograms, which can be processed as images. Choi et al. [3] demonstrated the effectiveness of Convolutional Neural Networks (CNNs) for various music tagging tasks, showing that CNNs can capture local temporal and harmonic patterns more effectively than traditional signal processing methods. A prominent architecture in this field is the Residual Network (ResNet), introduced by He et al. [4]. ResNet addresses the "vanishing gradient" problem in deep networks through the use of skip connections (residual blocks). ResNet architectures have also been adapted successfully to spectrogram-based audio tasks because their residual connections enable effective training of deep feature extractors for complex time-frequency patterns.

3.3 Semantic Text Analysis: Transformers and BERT

Lyrical content plays a crucial role in a song's emotional resonance and market appeal. Traditional Natural Language Processing (NLP) methods, such as Bag-of-Words, often fail to capture context and semantic nuance. The introduction of BERT (Bidirectional Encoder Representations from Transformers) by Devlin et al. [5] marked a paradigm shift in NLP. Unlike previous models, BERT utilizes a mechanism of self-attention to understand the bidirectional context of words in a sentence. For tasks requiring sentence-level comparisons, Reimers and Gurevych [6] proposed Sentence-BERT (SBERT). This modification uses siamese network structures to derive semantically meaningful sentence embeddings that can be efficiently compared using cosine similarity, making it a highly effective tool for encoding the semantic content of song lyrics.

3.4 Audio Feature Extraction for Tabular Modeling

While Deep Learning models like ResNet excel at processing raw spectrograms for classification, the optimization phase often requires a structured, interpretable state representation. To bridge the gap between continuous audio signals and the discrete feature space required by downstream

tasks, specific musical descriptors must be extracted. McFee et al. [7] introduced Librosa, a comprehensive Python library for audio and music signal analysis, which has become the industry standard for this task. Librosa employs Digital Signal Processing (DSP) algorithms to extract both low-level features (e.g., Spectral Centroid, Zero-Crossing Rate) and high-level rhythmic features (e.g., Tempo/BPM, Beat frames). These scalar values serve as explicit inputs for optimization algorithms, allowing for the manipulation of tangible attributes of the song rather than abstract neural weights.

3.5 Optimization via Reinforcement Learning and Tabular DL

While prediction is a well-established field, algorithmic optimization of music features is an emerging area of research. This task is often framed as a Markov Decision Process (MDP) that can be solved using Deep Reinforcement Learning (DRL). Early DRL breakthroughs such as Deep Q-Networks demonstrated that neural agents can learn effective decision policies in complex sequential environments [8]. Policy-gradient methods provide a direct approach for learning decision-making policies in sequential optimization problems. In particular, Proximal Policy Optimization (PPO) [9] stabilizes policy updates by limiting excessively large changes to the policy during training. Furthermore, to model the environment in such optimization tasks, a robust and computationally efficient function approximator is required. In this work, we employ a Multi-Layer Perceptron (MLP) as the surrogate model. MLPs are well-established universal approximators capable of modeling complex, non-linear relationships between structured input features and target outputs. Their low inference latency makes them particularly suitable for serving as simulated environments in Reinforcement Learning loops, where rapid feedback is essential for the agent's iterative training process.

4 General System Description

4.1 Background and System Objectives

The music industry today is highly saturated and competitive. While millions of songs are released to streaming platforms, the commercial performance of a track can be influenced by factors such as an artist's existing popularity and marketing resources, in addition to the

content and production characteristics of the song itself. Consequently, independent creators and producers often have to rely on intuition and trial-and-error during the music creation process, lacking objective tools to evaluate the quality of their work. The primary goal of the PopScore system is to solve this problem by providing an analytical, objective, and data-driven tool for evaluating and optimizing the commercial success potential of songs prior to their release. The system aims to make advanced artificial intelligence technologies accessible to the audio world, acting as a "virtual producer" that provides actionable feedback for sound improvement.

4.2 Implementation, Nature, and System Architecture

PopScore was designed as an accessible web application that integrates deep learning and reinforcement learning components in a single pipeline. The user interface is implemented with Gradio, while trained model weights and supporting assets are hosted on the Hugging Face Hub. The analytical pipeline consists of two main stages:

1. The Predictor Model: The system ingests a raw audio file (.mp3/.wav), the song's lyrics, and the intended release year. It converts the audio into a Mel-spectrogram (a time-frequency representation) and processes it using a convolutional neural network (ResNet50). Simultaneously, it analyzes the semantic meaning of the lyrics using a Natural Language Processing model (Sentence-BERT). Combining these data points allows the system to output a **"Popularity Score" (0-100)** and a market percentile, reflecting the song's content-based popularity potential while excluding direct artist-identity features.

2. The Optimization Model (AI Producer): Going beyond mere prediction, the system includes a Reinforcement Learning agent (PPO Agent) operating within a "virtual studio environment." The agent analyzes 8 acoustic features of the song (such as energy, loudness, danceability, etc.) and generates a **Studio Recommendations Report**. This report provides feature-level production recommendations (e.g., "Increase loudness by 1.5 dB" or "Decrease acousticness") intended to increase the song's estimated success potential and move its acoustic-feature profile toward patterns observed in successful tracks.

4.3 Target Audience

The system is designed for several key target audiences within the music industry:

- **Independent Artists (Indie Artists) and Emerging Creators:** Musicians lacking massive production budgets who seek objective, professional feedback on their demos before investing resources in releasing and marketing the track.
- **Music Producers and Mix/Mastering Engineers:** Studio professionals who can utilize the system as an AI-powered Copilot to validate artistic decisions, gain an algorithmic perspective on their mix, and perform final fine-tuning before wrapping up the production.
- **Record Labels and A&R (Talent Scouts):** Industry entities that receive thousands of demos annually. The system can serve as a smart filtering tool, identifying songs with high commercial potential and hit acoustics, even if the artist is completely unknown.

5 System Requirements

The system is defined by the following functional and non-functional requirements, designed to ensure accuracy, usability, and performance.

5.1 Functional Requirements

- **Input Ingestion:** The system shall allow users to upload audio files (e.g., MP3, WAV) and input song lyrics and release year.
- **Data Processing:** The system shall convert raw audio into spectrogram representations and encode lyrics into semantic embeddings for NLP processing.
- **Popularity Prediction:** The system shall calculate and display a predicted popularity score (0-100) based on the processed inputs.
- **Recommendation Engine:** Based on the analysis, the system shall provide textual recommendations for modifying interpretable acoustic features in order to improve the song's estimated popularity potential.

5.2 Non-Functional Requirements

- **Performance:** The inference time (from upload to result display) shall not exceed 30 seconds under normal load.
- **Accuracy:** The model shall strive to achieve a low Mean Absolute Error (MAE) when comparing predicted popularity scores against ground-truth data from streaming services.
- **Reliability:** The surrogate model should maintain a strong correlation with the target scoring function on the validation dataset, ensuring that the RL agent receives sufficiently reliable feedback.
- **Validity:** All feature modifications suggested by the Virtual Producer agent must remain within predefined valid ranges in normalized feature space, ensuring that no mathematically invalid values are presented to the user.
- **Latency:** To enable efficient Reinforcement Learning training, the surrogate environment (MLP Regressor) must perform a single inference step (state-to-reward calculation) in less than 50 milliseconds (0.05 seconds).

6 Description of the Solution

6.1 System Architecture

The solution is engineered as a dual-pipeline system designed to function sequentially. The architecture bridges the gap between high-dimensional unstructured data (audio/text) used for prediction, and structured tabular data used for optimization. The workflow consists of three main modules:

1. The Multimodal Predictive Module (Perception): A deep learning pipeline responsible for estimating the popularity score based on raw inputs.

2. The Optimization Module (Virtual Producer): A Reinforcement Learning (RL) loop responsible for suggesting feature modifications to improve the score.

3. The System Integration Module (Deployment): A cloud-connected, interactive web application for real-time user engagement.

6.2 Phase I: The Predictive Module

This module employs a "Late Fusion" strategy, processing each modality independently before combining them for the final regression task.

Audio Processing Branch:

Input: Raw MP3/WAV audio file.

Preprocessing: The audio is converted into a Mel-spectrogram, a time-frequency representation of the signal.

Model: We utilize ResNet-50, a deep Convolutional Neural Network (CNN). The model is trained from scratch (with `weights=None`) and modified to accept single-channel spectrogram inputs. This configuration allows the network to learn task-specific time-frequency patterns directly from the training data. The network produces a high-dimensional audio representation used by the final multimodal predictor.

Lyrics Processing Branch:

Input: Raw text of the song's lyrics.

Model: We utilize a Sentence-Transformer model, `all-MiniLM-L6-v2`, to encode the lyrics. The model produces a fixed-size semantic embedding that represents the overall textual content of the song and can be combined with the audio representation and release-year metadata.

Fusion and Regression:

- The embeddings from the audio branch and the text branch are concatenated with the normalized release year.
- The combined representation is fed into a Multi-Layer Perceptron (MLP) regressor to produce the final predicted popularity score ($P_{pred} \in [0, 100]$).

Advanced Loss & Score Calibration:

To prevent the model from collapsing toward the median score, we implemented a custom WeightedMSELoss function that exponentially increases the penalty for errors on extreme target values. The raw output then undergoes Quantile Mapping calibration against the observed Spotify popularity distribution. In the deployed demo, this calibration is followed by an application-level "Mastering Compensation Factor" intended to account for the possibility that users upload unfinished or unmastered demos. The resulting score ($P_{pred} \in [0, 100]$) is

presented as an estimate of content-based popularity potential rather than a measure of artist fame.

6.3 Phase II: The Optimization Module

Once a prediction is made, the system activates the optimization agent. Since manipulating raw spectrogram pixels is non-intuitive for music production, this module operates on interpretable, low-level audio features.

State Representation (Feature Extraction): The system constructs an 8-dimensional acoustic feature vector consisting of Danceability, Energy, Loudness, Speechiness, Acousticness, Instrumentalness, Liveness, and Valence. In the current lightweight inference pipeline, selected descriptors are estimated directly from the audio signal using Librosa-based signal processing, while features that are not directly estimated by this pipeline are initialized using predefined baseline values. The resulting vector constitutes the initial state (S_0) of the RL agent.

The Environment (Surrogate Model): To simulate the reaction of the market to changes in the song without re-running the heavy ResNet model, we employ a Multi-Layer Perceptron (MLP) regressor. This MLP acts as a lightweight surrogate model, trained (using oversampled extreme cases) to approximate the main predictor's output with high fidelity. Due to its low computational complexity, it serves as an efficient "Judge," providing immediate reward signals to the agent during the iterative optimization process.

The Agent (Proximal Policy Optimization): We implement a PPO agent, a state-of-the-art policy gradient method. The agent views the song as a state and possesses a discrete action space of 17 actions (8 actions to increase a specific feature by 0.05 in normalized feature space, 8 actions to decrease a feature by 0.05, and 1 "Stop" action). Its goal is to maximize the cumulative reward, defined as the positive difference in the popularity score predicted by the Surrogate MLP, while minimizing "distance penalties" to preserve the song's original identity.

6.4 Phase III: System Integration & User Interface

To transform the underlying algorithms into a usable product, the final phase focuses on deployment and user experience.

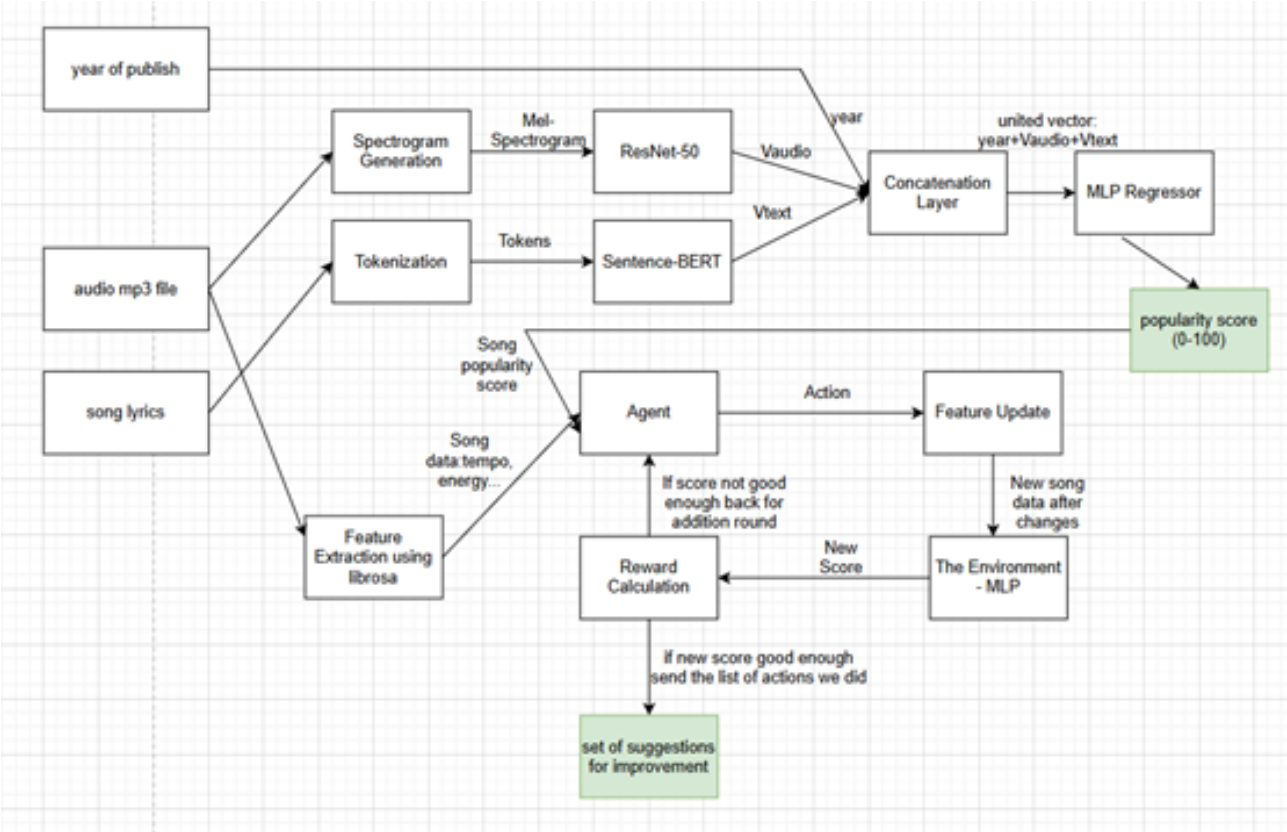
Instead of relying on local storage or heavy server processing, the system's architecture is

fully decoupled. All trained model weights and calibration datasets are hosted securely on the Hugging Face Hub. Upon initialization, the system pulls these assets directly into memory.

The frontend is built using the Gradio framework, allowing for a seamless, asynchronous web application. The interface provides users with an intuitive upload mechanism, real-time loading states, and comprehensive visual analytics—including dynamically generated Matplotlib spider charts (Radar plots) that compare the user's current track to the AI's "Ideal Hit" recommendations.

6.5 System Architecture Diagram:

The following diagram describes the entire system flow:



6.6 Algorithm Description: Optimization Process

The following pseudo-code describes the logic of the "Virtual Producer" agent during the inference phase (when a user uploads a song):

Algorithm 1 Song Optimization Agent (Inference)

Require: Audio file A
Ensure: Final optimized feature state S_{final} and recommendations R

$S_{initial} \leftarrow \text{ExtractAndNormalizeFeatures}(A)$
$S_{curr} \leftarrow S_{initial}$
$Steps \leftarrow 0$
while $Steps < \text{MAX\_STEPS}$ **do**
    $a \leftarrow \pi_\theta(S_{curr})$ ▷ Select action using the PPO policy
    if $a = \text{STOP}$ **then**
        break
    end if
    $S_{curr} \leftarrow \text{ApplyAction}(S_{curr}, a)$
    $S_{curr} \leftarrow \text{Clip}(S_{curr}, 0, 1)$
    $Steps \leftarrow Steps + 1$
end while
$S_{final} \leftarrow S_{curr}$
$R \leftarrow \text{Compare}(S_{initial}, S_{final})$
return S_{final}, R

7 Research and Development Process Description

The development of the PopScore system was conducted through a structured, iterative research and engineering process, transitioning from raw data exploration to a fully deployed cloud-based application. The process was divided into four primary phases:

7.1 Phase 1: Data Acquisition & Preprocessing

The foundation of the research required a robust, multi-modal dataset. We utilized a comprehensive Spotify dataset containing acoustic features, raw audio previews, and metadata.

- **Data Balancing:** Initial exploratory data analysis (EDA) revealed a heavy concentration of medium popularity scores. To reduce the tendency of the model to collapse toward the median, we balanced the training data through strategic oversampling of underrepresented extreme cases.
- **Feature Extraction:** We utilized the librosa library to process raw .mp3 and .wav files, converting them into 2D Mel-spectrograms (128 mel bands, padded/truncated to a fixed width of 1200 frames) to serve as visual input for our convolutional networks. Simultaneously, lyrical data was converted into dense semantic vectors using a pre-trained Sentence-BERT (all-MiniLM-L6-v2) model.

7.2 Phase 2: Deep Learning Predictive Modeling

The core predictive engine required an architecture capable of fusing completely different data modalities (Vision, NLP, and Tabular).

- **Multimodal Architecture:** We developed a custom PyTorch model (HitPredictor) featuring a modified ResNet50 backbone (adapted for 1-channel spectrograms) to extract deep audio features. These features were concatenated with the text embeddings and normalized release year, then passed through a Multi-Layer Perceptron (MLP) head to produce a normalized raw popularity score.

- **Custom Loss Function:** A major breakthrough in our research was the implementation of a custom WeightedMSELoss function. Standard MSE struggled with edge cases, so we designed a dynamic loss function that exponentially increases the penalty weight as the target popularity deviates from the median (0.5). This forced the model to learn the distinct acoustic signatures of extreme hits.

- **Score Calibration:** To map the model's raw output to realistic market expectations, we implemented a Quantile Mapping calibration technique against the real-world Spotify distribution. Furthermore, we applied a "Mastering Compensation Factor" to account for the fact that users upload raw demos lacking professional studio mastering.

7.3 Phase 3: The Optimization Engine (Reinforcement Learning)

To transition the system from a passive predictor to an active "Virtual Producer," we implemented a Reinforcement Learning (RL) pipeline.

- **The Surrogate Model:** Training an RL agent directly against the heavy ResNet model was computationally unfeasible. Therefore, we trained a lightweight SurrogateMLP that maps 8 core acoustic features (Danceability, Energy, Loudness, etc.) directly to a popularity score, achieving near-instant inference.

- **The Studio Environment:** We built a custom Gymnasium environment simulating a professional music studio. The observation space consists of the 8 acoustic features, and the action space allows incremental adjustments of ± 0.05 in normalized feature space.

- **Agent Training (PPO):** We trained a Proximal Policy Optimization (PPO) agent within this environment. To ensure the agent's recommendations remained musically logical, we engineered strict environment rules, including boundary constraints and a "distance penalty" that punishes the agent for aggressively destroying the song's original acoustic identity.

7.4 Phase 4: System Integration & Cloud Deployment (MLOps)

The final phase involved migrating the research codebase into a robust, scalable production environment.

- **Decoupled Architecture:** To improve portability, we migrated the trained model weights (.pth and .zip files) and required data assets to the Hugging Face Hub. Upon initialization, the application downloads the required assets through the Hugging Face Hub API and loads them into memory, reducing dependence on bundled local model files.
- **Interactive Frontend:** We engineered the user interface utilizing the Gradio framework. This allowed us to build a seamless, asynchronous web application directly from Python. The UI includes dynamic loading screens, real-time audio parsing, interactive spider charts (Radar plots) built with Matplotlib for visual analytics, and a neatly formatted Studio Recommendations Report.

8 Tools & Technologies

The system architecture is built upon a modern, open-source technology stack, meticulously selected to balance computational efficiency, developer ergonomics, and scalability. This toolset aligns with industry standards for developing and deploying AI-integrated web applications.

- **Programming Language: Python 3.9+**

  Python was selected as the core language for the entire backend and logic layer due to its dominance in the Machine Learning ecosystem, robust community support, and rich ecosystem of specialized libraries for both audio signal processing and deep learning.

- **Frontend & User Interface: Gradio (Deployed)**

1. **Gradio:** For the final deployed application, the user interface is rapidly prototyped and served using the Gradio framework. Gradio allows for seamless, pure-Python creation of interactive machine learning web applications. It provides built-in components for audio uploading, real-time loading states, and direct integration with Matplotlib for dynamic visual analytics (e.g., Radar plots).

- **Machine Learning & Deep Learning Libraries:** The core intelligence of the predictive and optimization engines relies on the following specialized Python libraries:
  - **PyTorch:** The primary deep learning framework utilized for constructing, training, and running the custom ResNet50 architecture and the Surrogate MLP. PyTorch was selected for its dynamic computation graph, which significantly simplifies the debugging of complex, multi-modal architectures.
  - **Hugging Face Ecosystem (Sentence Transformers & Hub):** Used for loading the pre-trained sentence embedding model for lyrical representation and for hosting trained model weights and calibration assets used by the deployed application.
  - **Librosa:** Used for audio loading, Mel-spectrogram generation, and extraction of selected low-level audio descriptors used by the inference pipeline.
  - **Stable-Baselines3 (SB3):** Utilized for the Reinforcement Learning pipeline, specifically providing a robust implementation of the Proximal Policy Optimization (PPO) algorithm used by the Virtual Producer agent.
  - **Scikit-learn:** Employed for data normalization (MinMaxScaler), preprocessing pipelines, and calculating standard evaluation metrics.

- **Development & Deployment Environment:**
  - **Google Colab Pro:** Model training and heavy data processing were conducted on Google Colab Pro to leverage NVIDIA T4/A100 GPUs, drastically accelerating the computation required for Deep Learning and RL agent training.
  - **Version Control:** The source code is version-controlled and collaboratively managed via GitHub.

9 Challenges

The development of the PopScore system involved several analytical, algorithmic, and engineering challenges. Addressing these challenges required iterative experimentation and careful decision-making regarding model accuracy, computational efficiency, interpretability, and practical usability.

- **Imbalanced Popularity Distribution and Prediction Collapse:** One of the main analytical challenges was the highly imbalanced distribution of popularity scores in the dataset. Most songs were concentrated around medium popularity values, while extremely successful hits and very low-popularity songs were relatively rare. As a result, initial models tended to minimize prediction error by producing scores close to the center of the distribution, leading to insufficient separation between strong and weak songs.

  To address this issue, we applied strategic oversampling of extreme popularity cases and introduced a custom WeightedMSELoss function that assigns higher penalties to prediction errors on songs whose true popularity scores are far from the median. The reasoning behind this decision was that standard MSE treats all prediction errors equally and therefore does not sufficiently encourage the model to learn rare but important extreme cases. In addition, Quantile Mapping calibration was applied as a post-processing step to spread the model outputs over a more realistic score range and align their distribution with the observed popularity distribution.

- **Integration of Heterogeneous Multimodal Data:** A major challenge was combining fundamentally different types of information: raw audio, free-form lyrics, and numerical metadata. These inputs differ significantly in structure, dimensionality, and semantic meaning.

  To solve this problem, we adopted a Late Fusion architecture. Each modality is processed independently using a model suited to its characteristics, and the resulting representations are combined only at a later stage. This approach was selected because direct integration of raw heterogeneous inputs would make the learning process more difficult and less structured.

- **Computational Cost of Reinforcement Learning:** Training a Reinforcement Learning agent directly against the full multi-modal predictor was computationally expensive and impractical. Reinforcement Learning requires a large number of interactions with the environment, and using the complete prediction pipeline at every step would significantly increase training time.

  To solve this problem, we trained a lightweight Surrogate MLP model that maps the structured acoustic features directly to an estimated popularity score. The Surrogate model provides fast feedback to the PPO agent and allows the optimization process to operate efficiently. The main consideration was to create a practical compromise between computational cost and the ability to evaluate many possible feature modifications.

- **Producing Meaningful Improvements Without Destroying the Original Song Identity:** Another challenge was preventing the RL agent from maximizing the predicted score through unrealistic or excessive changes. Without constraints, the agent could move toward feature combinations that are mathematically favorable but musically unreasonable.

  We addressed this issue by constraining normalized feature values to valid ranges, clipping normalized state values, limiting the number of optimization steps, and using a distance penalty that discourages the agent from moving too far from the original song state. In addition, the agent performs small incremental changes instead of large modifications.

  The main consideration behind these decisions was that the system should improve an existing song rather than transform it into a completely different one.

- **Integration of Independent Components into a Single Application:** A significant engineering challenge was integrating multiple independently developed components into a single end-to-end application. The system includes audio preprocessing, text processing, prediction, score calibration, acoustic feature extraction, RL optimization, recommendation generation, visualization, and user interface logic.

  The main technical challenge was maintaining compatibility between input formats, tensor dimensions, normalization methods, model weights, and execution order. To improve portability and simplify deployment, the trained model weights and required assets were

stored on the Hugging Face Hub and loaded automatically when the application starts. Gradio was selected to connect the Python inference pipeline directly to an interactive user interface.

10 Testing and Validation Strategy

To rigorously evaluate the performance and reliability of the PopScore system, we implemented a comprehensive statistical validation framework. We conducted empirical experiments on a balanced validation set of 2,808 unseen tracks to quantitatively evaluate both the predictive performance of our deep learning pipeline and the optimization behavior of our reinforcement learning agent.

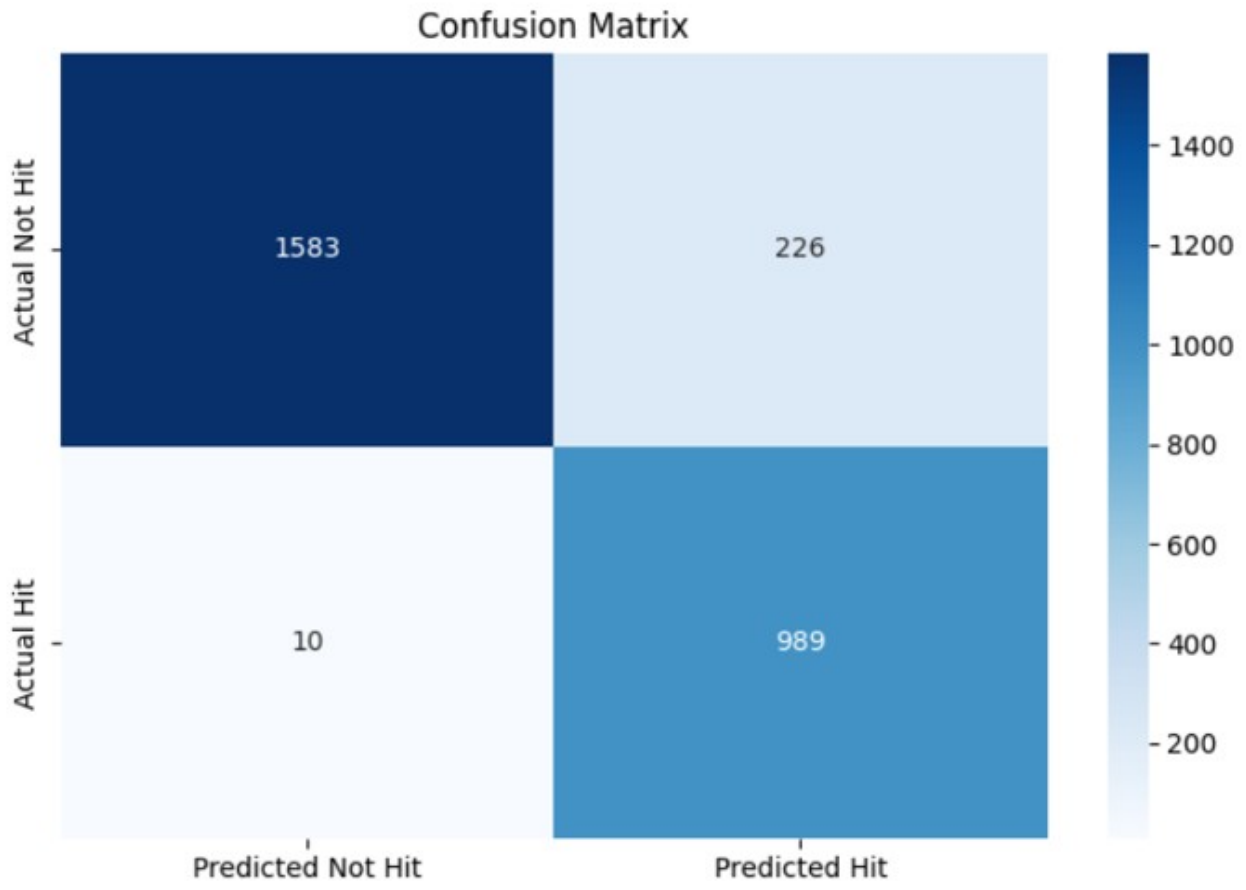
10.1 Predictive Model Evaluation (Hit Predictor)

We tested the multimodal predictive engine (ResNet-50 + Sentence Transformer) on the unseen validation set to evaluate its ability to capture popularity-related patterns across the input modalities.

Ranking & Regression Analysis: The model achieved a **Spearman Correlation of 0.824** ($p < 0.0001$). This strong monotonic relationship indicates that the model preserves a meaningful rank ordering between lower- and higher-popularity tracks on the evaluated dataset. The Mean Absolute Error (MAE) stood at 19.06 popularity points.

Classification Performance: To evaluate real-world utility, we framed the prediction as a binary classification task, defining any score above 60.0 as a "Hit." The model achieved an **overall accuracy of 92%** and a weighted F1-score of 0.92.

Confusion Matrix Analysis: As seen in the confusion matrix below, the model correctly identified 989 out of 999 tracks labeled as hits in the validation set, resulting in a **recall of 0.99** for the "Hit" class, with only 10 false negatives.



10.2 Distribution Alignment and Calibration (K-S Test)

A critical requirement for the system was ensuring that the predicted scores map realistically to the actual market distribution, rather than collapsing to an artificial median. To achieve this, we applied a Quantile Mapping calibration technique.

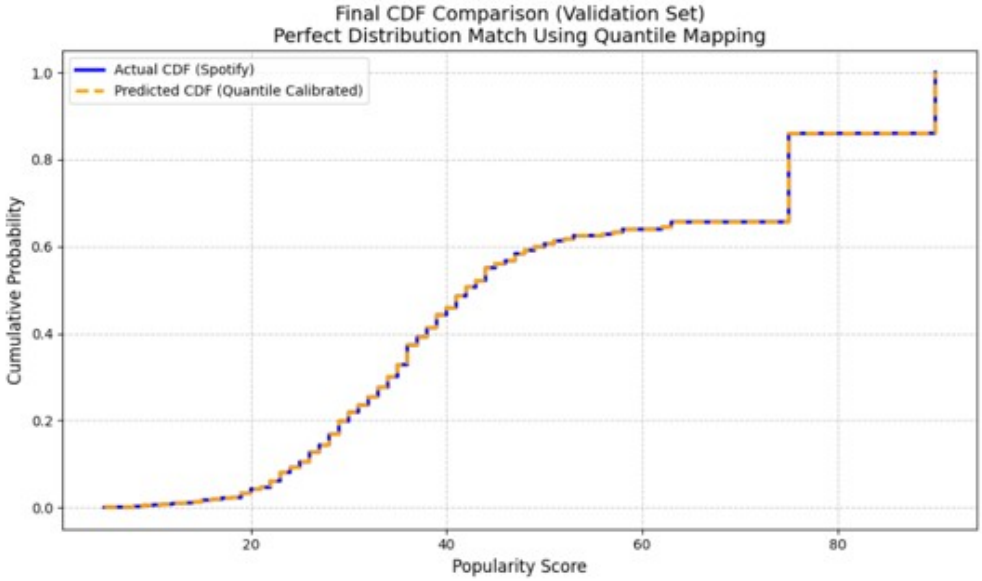
To evaluate the distributional alignment, we employed the **Kolmogorov-Smirnov (K-S) Test** to compare the empirical cumulative distributions of the calibrated predictions and the target popularity scores.

Results: The K-S test yielded a statistic of $D = 0.0003$ with a p-value of 1.0.

Conclusion: The very small K-S statistic indicates extremely close empirical alignment between the calibrated prediction distribution and the target popularity distribution. The high p-value indicates that the test does not provide evidence for a statistically significant difference between the two empirical distributions.

```
K-S Statistic (D): 0.0003 <-- (Should be much lower now!)
P-value: 1.00000e+00

============================================================
📊 2. Chi-Squared Goodness of Fit Test (χ²)
============================================================
Chi-Squared Statistic (χ²): 0.0000
P-value: 1.00000e+00
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:579: RuntimeWarning: ks_2samp: Exact calculation unsuccessful. Switching to method=asymp.
  res = hypotest_fun_out(*samples, **kwds)
```



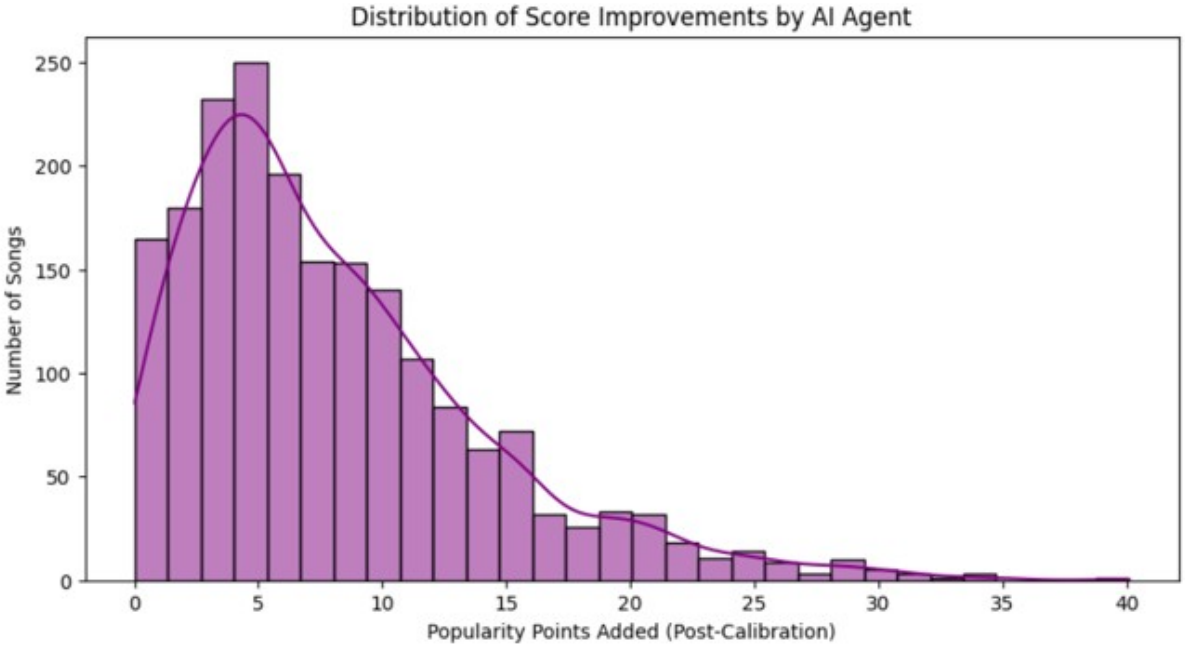
10.3 Virtual Producer Validation (RL Agent Performance)

To validate the Reinforcement Learning agent (the "Virtual Producer"), we evaluated its ability to safely and effectively increase a song's popularity potential without destroying its acoustic identity. We executed the agent over the unseen tracks and recorded the improvement metrics.

Success Rate: The agent found optimization trajectories associated with a positive surrogate-estimated score improvement for **71.1%** of the tested songs.

Improvement Metrics: On average, the agent achieved a surrogate-estimated improvement of **+5.74 popularity points** per track. The maximum estimated improvement observed for a single track was **+40.09 points**. **Behavioral Verification:** The agent utilized the full optimization horizon on average, indicating active modification of the feature space. This behavior also suggests that future work should further investigate the stopping policy and the trade-off between score improvement and intervention length.

The distribution of these score improvements is visualized below and exhibits a right-skewed shape, indicating that most improvements were moderate while a smaller number of tracks showed larger estimated gains.



10.4 Meeting Project Metrics and Success Criteria

The evaluation results demonstrate that the project achieved its main predictive, optimization, and deployment objectives. The following results summarize the primary validated outcomes:

High Predictive Accuracy: We aimed for reliable classification of tracks labeled as hits. An overall accuracy of 92% and a recall of 0.99 for the hit class indicate strong classification performance on the evaluated validation set.

Actionable Optimization: A core objective was to provide useful and interpretable optimization guidance. The RL agent found positive surrogate-estimated optimization paths for 71.1% of the evaluated tracks, supporting its intended role as an AI-assisted decision-support tool for music producers.

Statistical Validity: We required the AI's output to reflect real-world market distributions to ensure user trust. A K-S test p-value of 1.0, together with a very small K-S statistic, indicates extremely close distributional alignment on the evaluated data.

Seamless Usability: We deployed an interactive web application using Gradio and Hugging Face-hosted assets. The application performs the required multimodal inference and optimization pipeline without requiring local GPU hardware from the end user.

10.5 Conclusions

The project successfully achieved its main objective of developing an end-to-end system that combines song popularity prediction with actionable optimization recommendations. The evaluation results demonstrate that the multimodal model is capable of learning meaningful relationships between audio characteristics, lyrical content, temporal information, and popularity. The Reinforcement Learning module also demonstrated that a prediction system can be extended into an optimization framework by operating in an interpretable acoustic feature space. The results show that the agent was able to identify potential improvement paths for a substantial portion of the evaluated songs.

At the same time, several limitations remain. The optimization process currently operates in feature space and does not directly modify the audio waveform. Therefore, the recommended changes represent predicted improvement directions rather than guaranteed improvements to the final mastered audio. Future work should focus on improving feature extraction accuracy, validating the recommendations through human listening tests and professional audio evaluation, and integrating the recommendation engine with DSP-based audio processing tools that can automatically apply the suggested modifications.

Overall, the project goals were achieved: the system provides popularity prediction, relative market positioning, interpretable optimization recommendations, and an accessible web-based user interface.

11 AI and Supporting Tools

The following tools assisted us during the documentation, design, and presentation stages of the project: **Gemini:** Assisted in synthesizing technical concepts, drafting high-level academic explanations, and structuring the project's documentation and theoretical background.

Draw.io: Employed for designing detailed system architecture diagrams and flowcharts, visually mapping the end-to-end pipeline from data ingestion to the optimization output.

12 References

1. Castelli, E. (2021). Hit Song Prediction system based on audio and lyrics embeddings. Master's Thesis, Politecnico di Milano.

2. Oramas, S., Nieto, O., Barbieri, F., & Serra, X. (2017). Multi-modal music genre classification with audio, text and images. In Proceedings of the 18th ISMIR Conference, 168-175.

3. Choi, K., Fazekas, G., Sandler, M., & Cho, K. (2017). Convolutional recurrent neural networks for music classification. In 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2392-2396.

4. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 770-778.

5. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint arXiv:1810.04805.

6. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing.

7. McFee, B., Raffel, C., Liang, D., Ellis, D. P., McVicar, M., Battenberg, E., & Kelley, O. (2015). librosa: Audio and music signal analysis in python. In Proceedings of the 14th python in science conference, 18-25.

8. Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529-533.

9. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347.

Maintenance Guide

The purpose of this maintenance guide is to ensure the longevity and continuity of the PopScore project. It provides future developers, researchers, and maintainers with the necessary technical instructions to deploy, modify, and upgrade the system infrastructure and algorithms seamlessly.

Recommended Development Environment (Google Colab)

The most efficient and recommended way to maintain, test, or retrain the PopScore models is by using **Google Colab.** This cloud-based approach eliminates complex local hardware configurations and dependency issues, providing immediate access to required resources.

Prerequisites:

- A Google Account with access to Google Drive (for saving datasets or new model weights).
- Access to the project's GitHub repository.

Execution Steps in Colab:

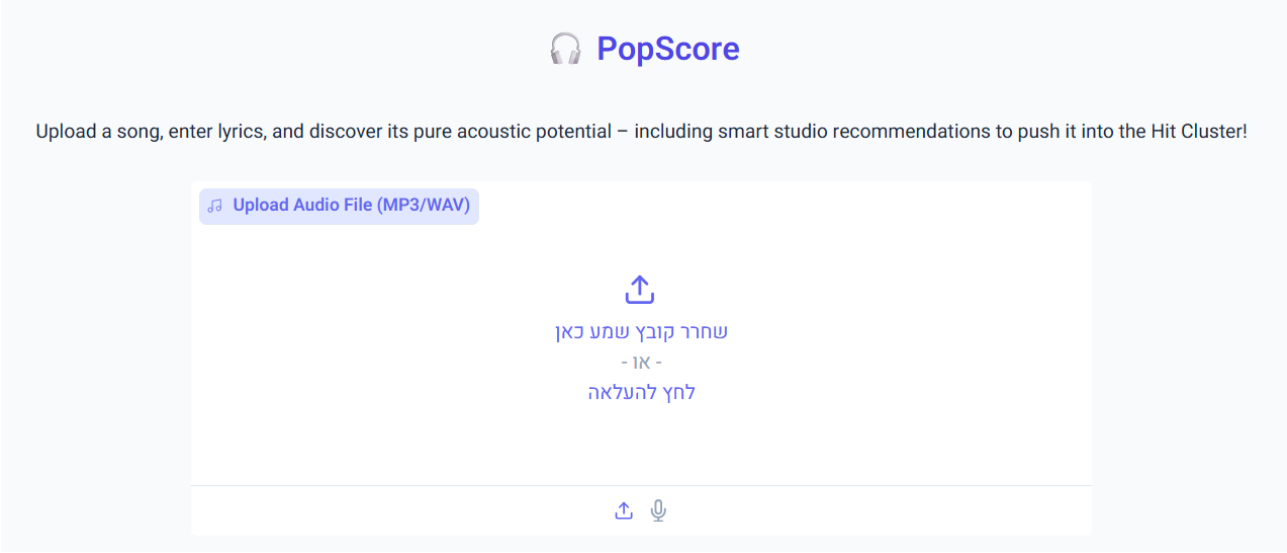
- 1. Open a new or existing Google Colab Notebook.
- 2. Go to Runtime -> Change runtime type and select a **T4 GPU** (or higher). This is highly recommended to drastically reduce the time required for audio Mel-spectrogram extraction and neural network inference.
- 3. Clone the project repository directly into the Colab environment:
  git clone https://github.com/aradharush/Song-Popularity-Prediction-System
  cd Song-Popularity-Prediction-System
- 4. Install the required dedicated libraries. Note: Colab already comes pre-installed with many essentials (like numpy, pandas, torch, and the underlying ffmpeg OS package required for audio decoding). You only need to install the specific missing packages:
  !pip install librosa stable-baselines3 sentence-transformers gradio huggingface_hub
- 5. Open the **App_Run.ipynb** notebook in Google Colab and run all cells sequentially using Runtime → Run all. Because `share=True` is enabled in the Gradio launch parameters, running the notebook will generate a temporary public Gradio URL that allows you

to interact with the UI directly from your browser while the application runs on Google's servers.

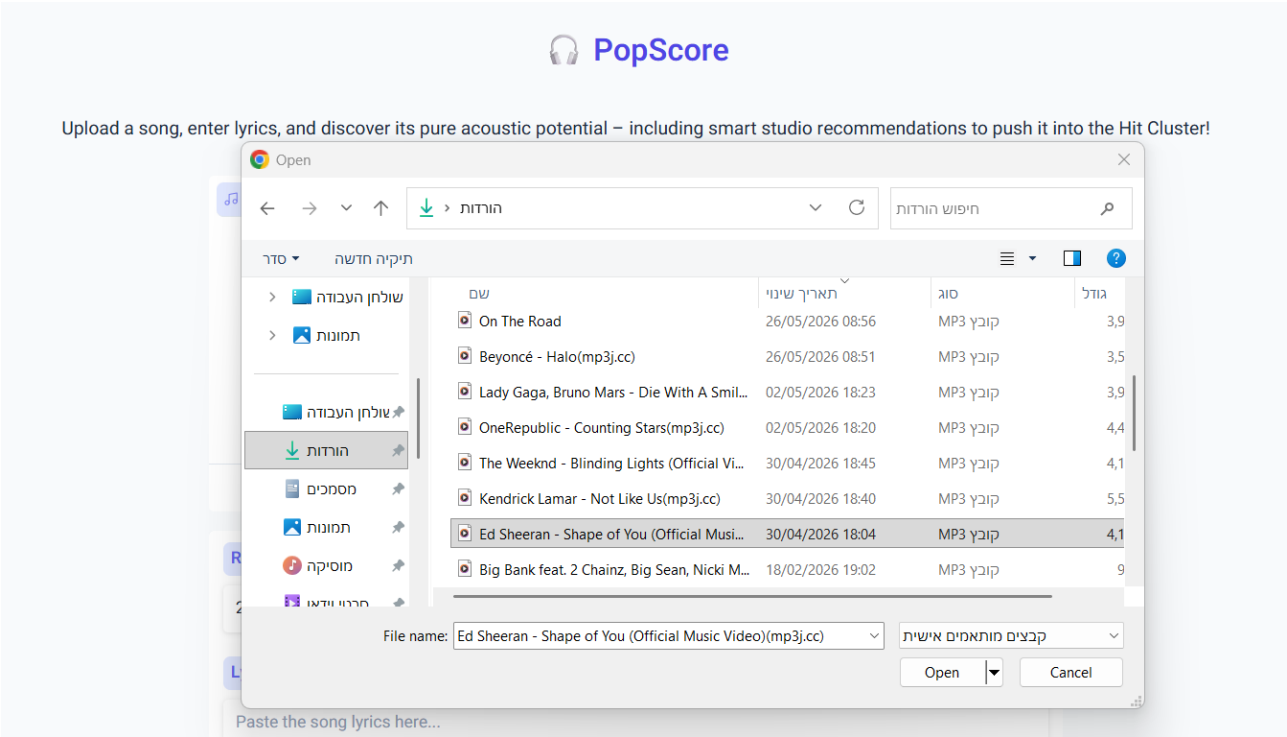
User Guide

Analyzing a new track begins on the app's home screen. Once launched, you will be prompted to enter the song's details. Follow the steps below to complete the setup and begin your analysis:

Step 1: Upload an Audio File

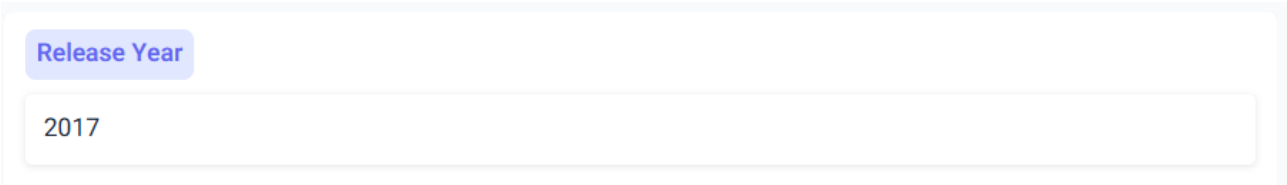


Click the upload area to select an audio file from your device, or drag and drop the file into the box.



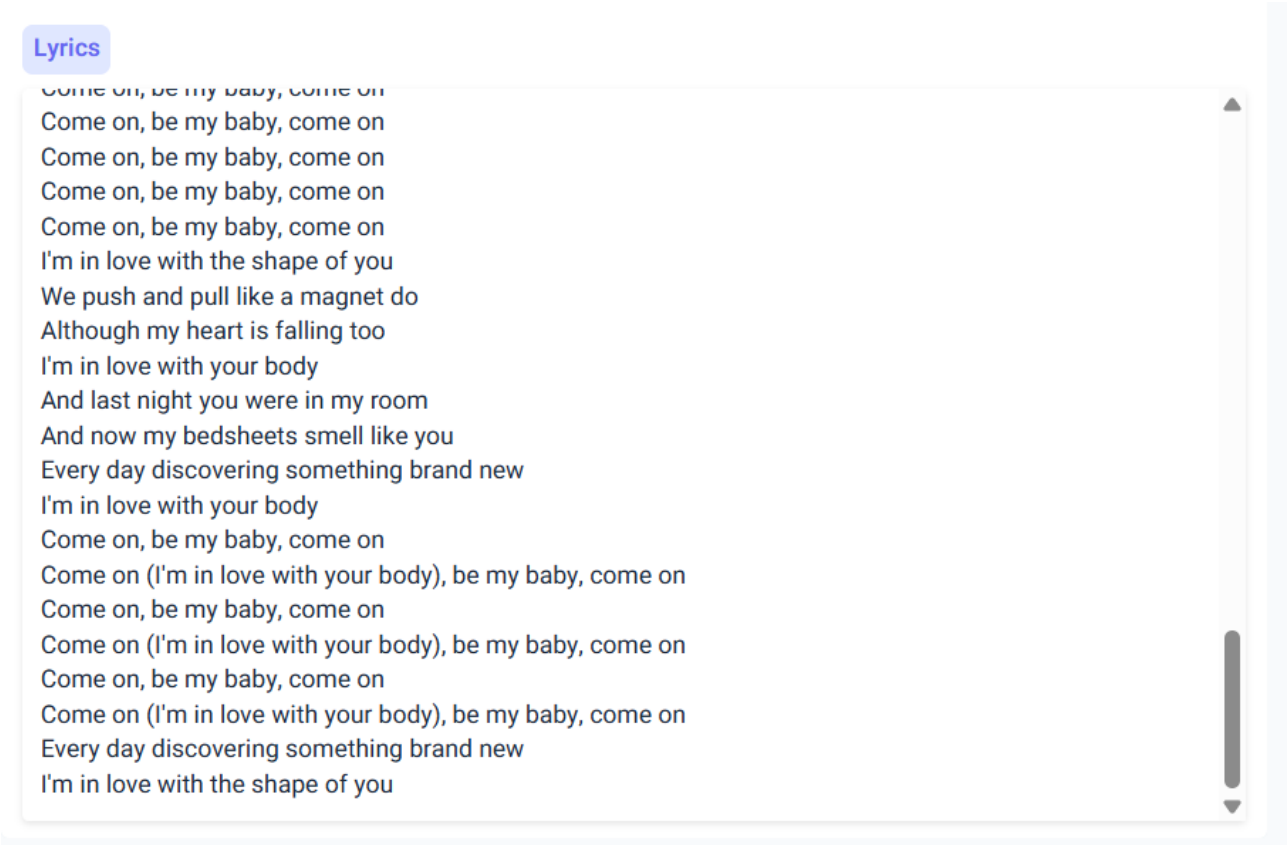
Select the file from your file system and click Open.

Step 2: Enter the Release Year



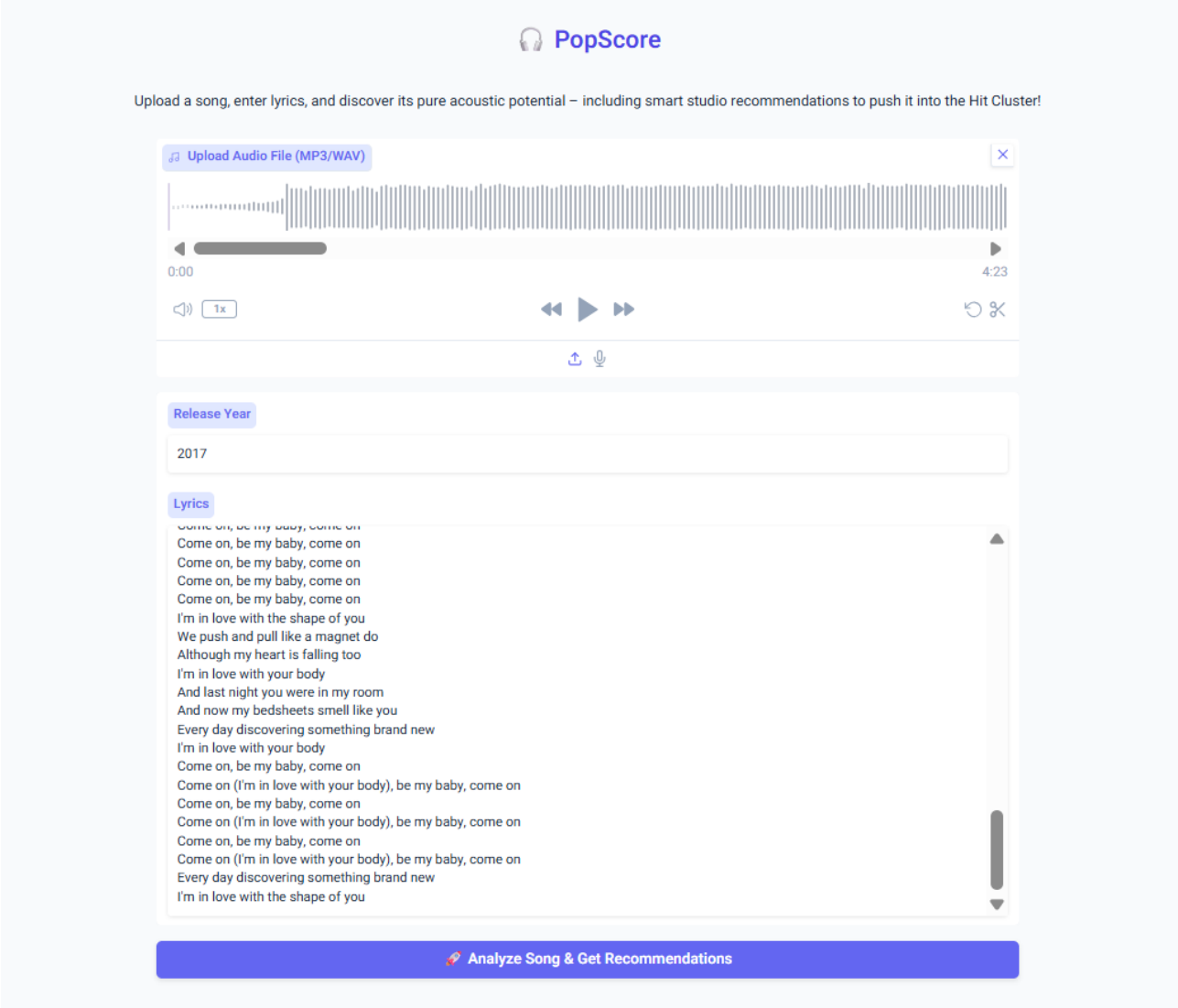
Enter the intended release year of the song.

Step 3: Enter the Song Lyrics



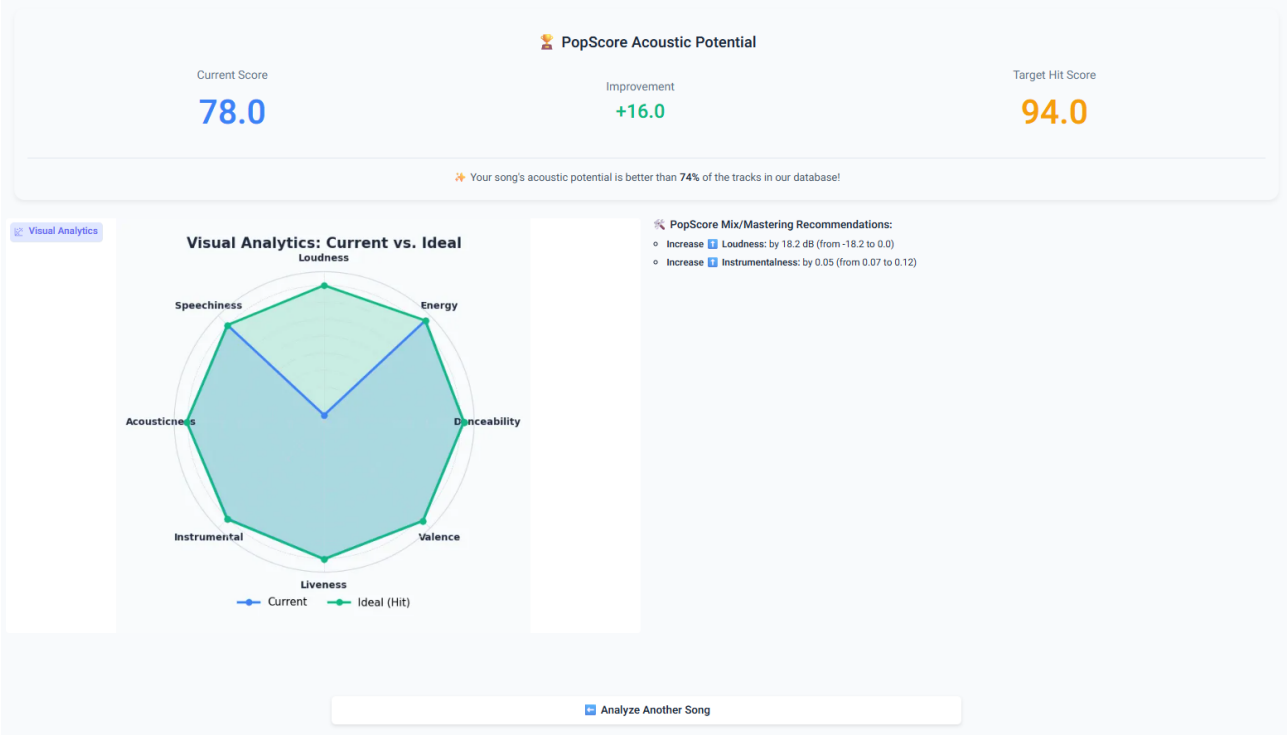
Paste or type the song lyrics into the text box.

Step 4: Analyze the Song



After completing the first three steps, the home page should look similar to the example shown in the figure. Ensure that all three input fields are completed, and then click the "Analyze Song & Get Recommendations" button.

Step 5: Review the Results



After the system completes its analysis, you will see the song's current score and its estimated optimization target under "Target Hit Score." To reach this target, follow the detailed steps in the "PopScore Mix/Mastering Recommendations" section. You can also explore the before-and-after feature differences visually via the spider chart on the left. Click the "Analyze Another Song" button to return to the home page and start a new analysis.